Fraud Detection Case Study: Identifying Credit Card Fraudulent Transactions

Introduction:

In this case study, we will explore a fraud detection problem using a dataset containing information about credit card transactions. The goal is to build a model that can accurately classify transactions as fraudulent or benign based on various features associated with each transaction.

Dataset Description:

The dataset consists of the following columns:

Column Name	Description
step	Maps a unit of time in the real world. 1 step = 1 hour of time.
Customer	Unique customer ID associated with each transaction.
zipCodeOrigin	The zip code of the transaction's origin/source.
Merchant	The unique ID of the merchant involved in the transaction.
zipMerchant	The zip code of the merchant.

Column Name	Description
Age	Categorized age of the customer:
	0: <= 18
	1: 19-25
	2: 26-35
	3: 36-45
	4: 46-55
	5: 56-65
	6: > 65

Column Name	Description
	U: Unknown
Gender	Gender of the customer:
	E: Enterprise
	F: Female
	M: Male
	U: Unknown
Category	Category of the purchase.
Amount	The amount of the purchase.

Column Name	Description
Fraud	Target variable: 1 if transaction is fraudulent, 0 if benign.

Objective:

The main objective of this case study is to develop a predictive model that can accurately identify fraudulent credit card transactions based on the provided dataset.

Key Steps for consideration:

Data Preprocessing:

- Handle missing values, if any, using appropriate techniques.
- Encode categorical variables (Age, Gender) using techniques such as one-hot encoding.
- Split the dataset into features (independent variables) and the target variable (Fraud).

Exploratory Data Analysis (EDA):

- Explore the distribution of the target variable (Fraud) to understand the class imbalance.
- Analyze the distribution of features and their relationships with the target variable.
- Visualize trends, patterns, and potential outliers in the data.

Feature Engineering:

- Create new relevant features if possible, such as the time of day from the 'step' column.
- Normalize or scale numerical features as needed.

Model Selection:

- Choose appropriate machine learning algorithms for fraud detection, such as Random Forest, Gradient Boosting, Logistic Regression, or Neural Networks.
- Set up a suitable evaluation metric considering the class imbalance, such as F1-score or Area Under the ROC Curve (AUC-ROC).

Model Training:

- Split the dataset into training and testing subsets.
- Train the selected models on the training data.

Model Evaluation:

- Evaluate the models using the chosen evaluation metric on the test data.
- Compare the performance of different models and select the best-performing one.

Model Interpretation:

- Interpret the model's predictions and identify the key features influencing fraud detection.

Handling Imbalance:

- Implement techniques to address class imbalance, such as oversampling, under sampling, or using synthetic data generation (SMOTE).

Fine-tuning and Optimization:

- Optimize hyperparameters of the selected model for better performance.

Database Integration:

- Integrate a relational database (such as MySQL, PostgreSQL, or SQLite) to store transaction data securely.
- Create functions to:
- Insert raw transaction data from the web application.
- Retrieve stored records for model retraining.
- Track the number of new entries and trigger retraining when a defined threshold is reached.
- Design a structured table to hold preprocessed and clean transaction data for consistent reuse.

Web Application:

- Develop a backend service to handle input processing and fraud prediction.

- Build a user-friendly frontend to collect input fields with proper validation and error checking to ensure data accuracy and completeness.
- Display prediction results, including the likelihood or probability of fraud.
- Automatically send submitted data to the backend database for Retraining.

Retraining Mechanism:

- Fetch the most recent data stored in the database.
- Apply consistent preprocessing techniques as used in initial training.
- Split the data into training and testing sets.
- Retrain the predictive model using updated data once a threshold of new records is met.
- Replace the deployed model with the newly trained version to maintain accuracy over time.

Key deliverables:

- Push the final model code to a Github and share the link.
- Create a PowerPoint explaining the steps you have considered and why.
- Create recommendations and key outcomes which can be presented to the business teams.